Clustering and Initialization

Statistical Methods for Machine Learning
A.Y. 2025/26

Mohammed Vohra

June 4, 2026

1 Introduction

This report studies how the starting point of the k-means algorithm affects the quality of clustering, based on the paper *k-means++: The Advantages of Careful Seeding* by Arthur and Vassilvitskii (SODA 2007). The standard k-means algorithm is known to be sensitive to initialization and can get stuck in bad solutions depending on where it starts.

Clustering is an unsupervised learning problem where the goal is to group similar data points together. The k-means algorithm does this by minimizing inertia — the sum of squared distances from each point to its cluster center. The problem is non-convex, which means the algorithm can end up in different solutions depending on the starting point. Two runs of the same algorithm on the same data can produce very different results.

The k-means++ algorithm fixes this by choosing the starting centers more carefully, spreading them across the data instead of picking them randomly. The goal of this project is to implement both algorithms from scratch and show through experiments when and why k-means++ does better.

2 Datasets

Three datasets were used. Since clustering is unsupervised — there are no labels involved — no train/test split was applied. The inertia is computed on the full dataset, which is the standard evaluation approach for clustering.

2.1 Synthetic Separated Dataset

A synthetic dataset was generated using Gaussian blobs with 300 points and 4 well-separated clusters (`cluster_std=0.8`), using `sklearn.datasets.make_blobs`. This dataset was chosen because the clusters are clearly defined, making it a good baseline to study the effect of initialization when the problem is straightforward. Figure 1 shows the dataset.

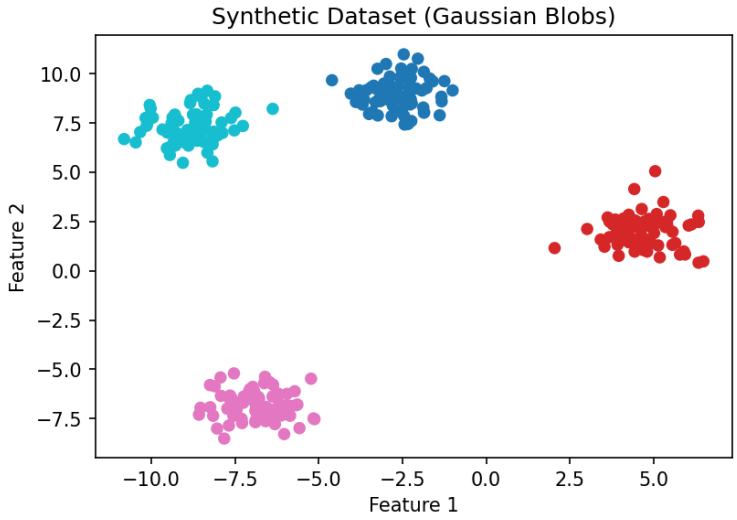

Figure 1: Synthetic dataset with 4 well-separated Gaussian clusters.

2.2 Synthetic Overlapping Dataset

A second synthetic dataset was generated with the same setup but with a higher spread (`cluster_std=2.5`), so the clusters overlap significantly. This was chosen to test what happens when the problem is harder and the cluster boundaries are not clear. Figure 2 shows the dataset.

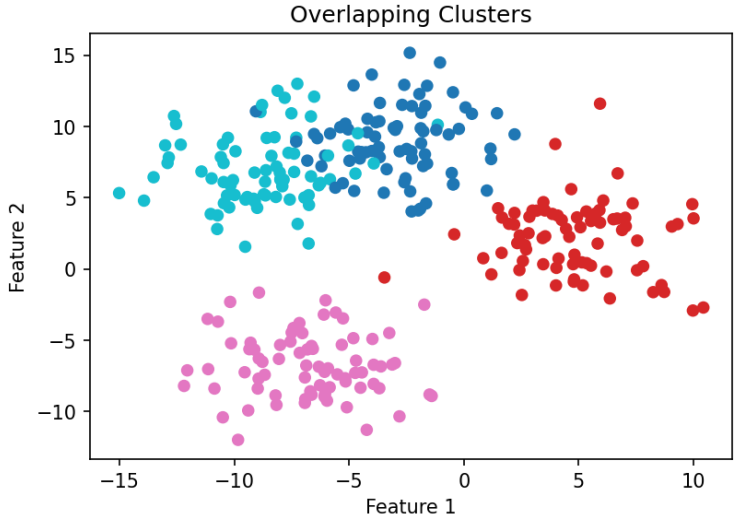

Figure 2: Synthetic dataset with 4 overlapping Gaussian clusters.

2.3 MNIST Dataset

A subset of 3000 samples from the MNIST handwritten digit dataset was used as the real-world dataset, loaded using `sklearn.datasets.fetch_openml`. Each image has 784 pixel

features, normalized to $[0, 1]$. MNIST has 10 digit classes, so $k = 10$ was a natural choice. This dataset was picked because it is high-dimensional and represents a realistic, complex clustering problem. The subset size of 3000 was chosen to keep the experiments fast enough to run multiple trials.

3 Implementation

Both algorithms were written from scratch using only NumPy. No clustering libraries were used.

3.1 K-means

The k-means algorithm works as follows:

1. Pick k random data points as the starting centers
2. Assign each point to the nearest center using Euclidean distance
3. Move each center to the mean of its assigned points
4. Repeat steps 2 and 3 until the centers stop moving

The main problem with this approach is step 1. If the random starting points are all in the same area, the algorithm gets confused and produces bad clusters.

3.2 K-means++

K-means++ only changes the initialization step. Instead of picking centers randomly, it spreads them out:

1. Pick the first center randomly
2. For each next center, pick a point with probability proportional to its squared distance from the nearest existing center:

$$P(x) = \frac{d(x)^2}{\sum_{x'} d(x')^2} \tag{1}$$

   where $d(x)$ is the distance from point x to the nearest already-chosen center.

3. Run standard k-means from these starting centers

This way, the starting centers are spread across the data, which reduces the chance of getting stuck in a bad solution. The paper proves that the expected cost of k-means++ is at most $O(\log k)$ times the optimal cost.

3.3 Inertia

Inertia measures how tight the clusters are — lower is better:

$$\text{Inertia} = \sum_{j=1}^{k} \sum_{x \in C_j} \|x - \mu_j\|^2 \tag{2}$$

where C_j is the set of points in cluster j and μ_j is the center of that cluster.

4 Experiments and Results

4.1 Synthetic Separated Dataset

Both algorithms were run for 50 trials with different random seeds. Figure 3 shows how the inertia values are distributed across trials.

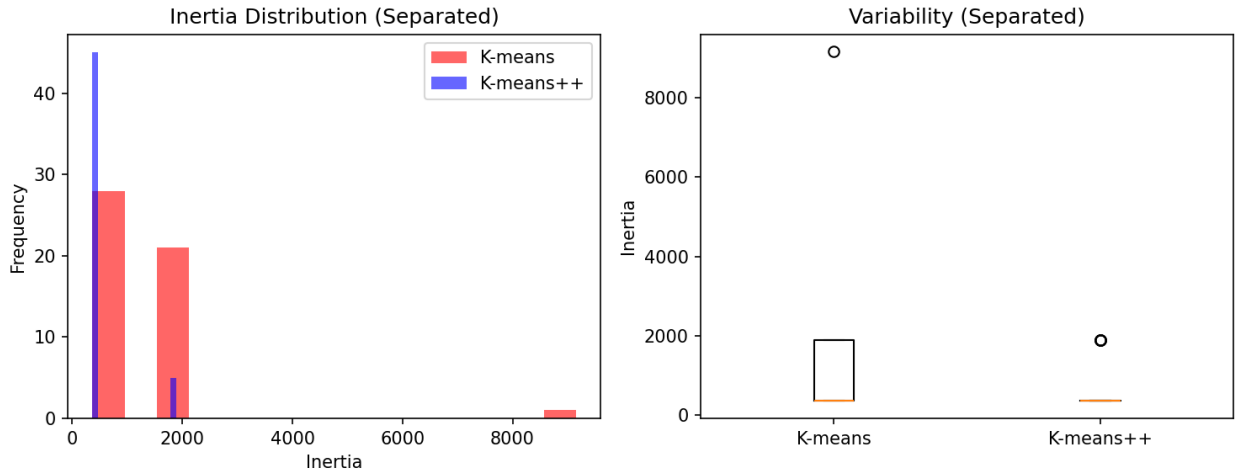

Figure 3: Inertia distribution and variability on the synthetic separated dataset (50 trials).

Algorithm	Mean Inertia	Std
K-means	1178.71	1363.12
K-means++	514.85	457.13

Table 1: Results on synthetic separated dataset (50 trials)

K-means++ gets more than twice lower inertia on average, and is much more consistent across trials. The histogram shows that k-means sometimes gets stuck at very high inertia (up to 9000), while k-means++ almost always finds a good solution.

Figure 4 shows what this looks like visually. In this particular run, k-means merged two natural clusters into one because of a bad starting point, while k-means++ correctly found all four clusters.

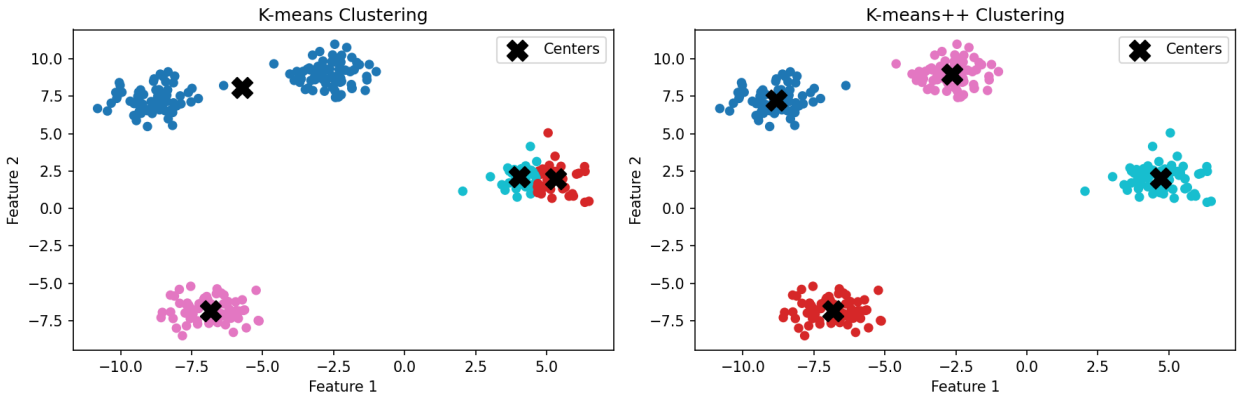

Figure 4: Visual comparison of k-means vs k-means++ on the synthetic separated dataset.

4.2 Synthetic Overlapping Dataset

50 trials were run on the overlapping dataset. Figure 5 shows the results.

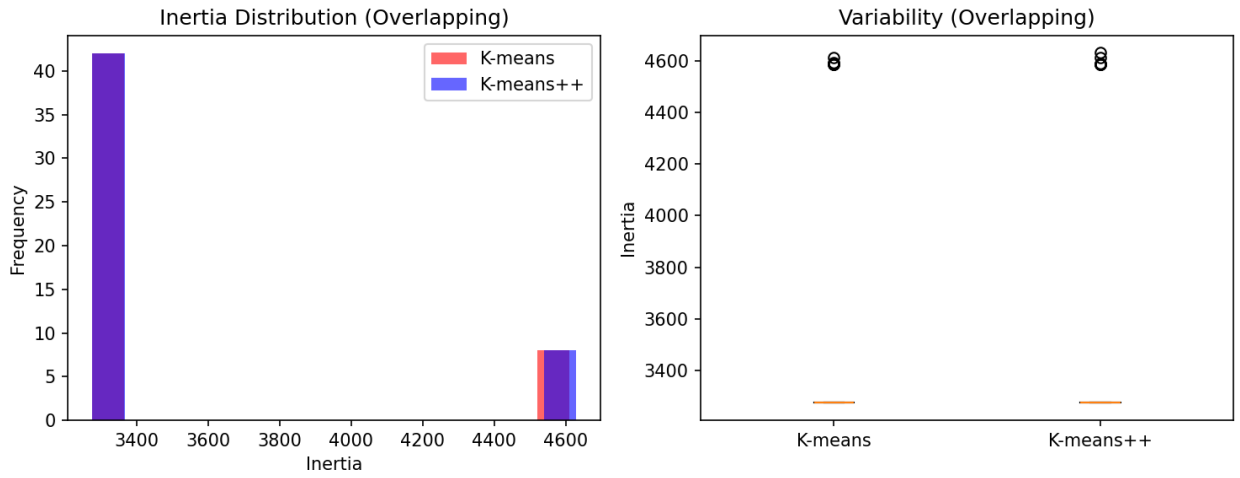

Figure 5: Inertia distribution and variability on the overlapping dataset (50 trials).

Algorithm	Mean Inertia	Std
K-means	3486.41	482.23
K-means++	3487.04	483.90

Table 2: Results on synthetic overlapping dataset (50 trials)

When clusters overlap, both algorithms perform almost the same. The smarter initialization does not help much here because the cluster boundaries are not clear — the problem is hard regardless of where you start.

4.3 MNIST Dataset

20 trials were run on the MNIST subset. Figure 6 shows the results.

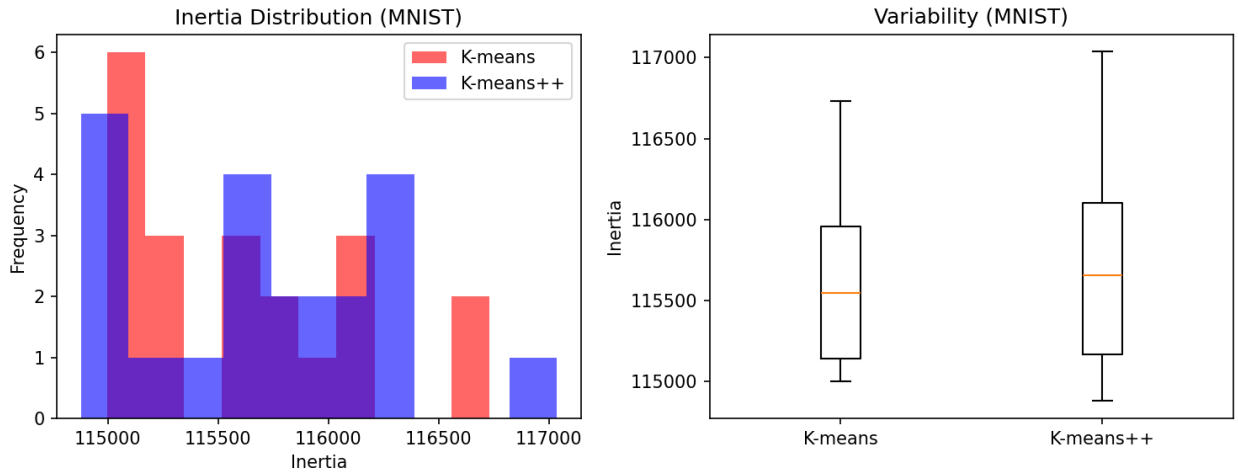

Figure 6: Inertia distribution and variability on MNIST (20 trials).

Algorithm	Mean Inertia	Std
K-means	115606.31	517.77
K-means++	115704.07	559.32

Table 3: Results on MNIST dataset (20 trials)

On MNIST, both algorithms give similar results. The data is complex and high-dimensional (784 features), which makes clustering hard for both methods. The inertia values are much larger than the synthetic case simply because there are many more features.

5 Discussion

The results show a clear pattern. K-means++ works much better than k-means when the clusters are well separated and easy to identify. In that case, starting from spread-out centers makes a big difference — the algorithm finds better solutions more consistently.

When the clusters overlap or the data is complex, both algorithms end up doing about the same thing. This makes sense because when the cluster structure is not clear, even a good starting point does not help much. In those situations, other choices — like how many clusters to use, or whether to reduce the dimensionality first — probably matter more than initialization.

One limitation of this study is that we always used the true number of clusters k. In practice, you usually do not know the right value of k, and choosing it incorrectly would affect both algorithms equally.

6 Conclusion

This project showed that initialization matters a lot for k-means, but mainly when the data has a clear cluster structure. K-means++ is a simple fix that works surprisingly well in those cases — it roughly halved the mean inertia on the separated synthetic dataset and made the results much more consistent across runs.

For harder problems like overlapping clusters or high-dimensional real-world data, the two algorithms perform similarly. This suggests that k-means++ is most useful when the data is well-structured, and that other aspects of the clustering pipeline become more important when the problem is harder.

References

[1] David Arthur and Sergei Vassilvitskii. *k-means++: The Advantages of Careful Seeding*. In Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 2007.

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.